

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

ОТЧЕТ

«АРМ администратора фитнес-клуба»

По дисциплине «Основы алгоритмизации и программирования»

Выполнили студенты группы: ИВТ-25-16

Фёдоров Марк Максимович, Мичков Роман Валерьевич

Принял: Доц. Полякова О. А.

Пермь 2026

СОДЕРЖАНИЕ

1. Введение
2. Постановка задачи
3. Анализ предметной области
4. Выбор средств разработки
5. Проектирование программного обеспечения
6. Описание UML-диаграммы
7. Реализация программного средства
8. Основные задачи разработки и принятые решения
9. Тестирование программного средства
10. Заключение
- 11.Список использованных источников

1. Введение

В условиях роста популярности фитнес-услуг и увеличения числа клиентов спортивных клубов возникает необходимость в автоматизации ключевых бизнес-процессов: учёта клиентов, расписания тренировок, контроля посещаемости, расчёта выручки и управления доступом. Многие существующие решения требуют постоянного подключения к интернету, ежемесячной подписки и передачи данных на внешние серверы, что создаёт дополнительные расходы и риски.

Целью данной работы является разработка полностью автономного оффлайн-приложения «GymOfflineQt» для фитнес-клуба, которое обеспечивает регистрацию пользователей с разными ролями (администратор, тренер, клиент), ведение расписания тренировок, фиксацию посещений, учёт продажи абонементов и разовых занятий, контроль доступа по персональному ID, а также аналитику выручки и популярности услуг. Приложение не требует интернета и хранит все данные локально в JSON-файле. Разработанная система реализована на языке C++ с использованием фреймворка Qt (версия 5.12.12), модуля Qt Widgets для графического интерфейса и стандартных контейнеров для работы с данными.

2. Постановка задачи

В ходе выполнения работы требовалось разработать настольное приложение, обеспечивающее выполнение следующих функций:

- регистрация и вход пользователей с ролями «Администратор», «Тренер», «Клиент»;
- ведение списка пользователей с полями: логин, пароль, роль, уникальный ID (для клиентов);
- для клиентов – покупка абонементов на 1, 3, 6 или 12 месяцев, просмотр остатка тренировок (групповых и персональных), история покупок;
- запись клиента на групповое или персональное занятие с выбором тренера, даты и времени;
- возможность переноса тренировки клиентом на другое время с проверкой пересечений;
- для тренера – добавление тренировок в график, учёт оборудования (добавление, удаление, изменение статуса и зала);
- для администратора – фиксация посещений (списание разовой оплаты, запись в историю), просмотр аналитики (выручка, общее число посещений, популярность занятий, отток клиентов), контроль доступа по ID клиента (проверка активного абонемента или наличия будущих тренировок);
- удаление записей (посещений, покупок, пользователей) только после подтверждения и при выделении строки в таблице;
- сохранение всех данных (пользователи, посещения, покупки, тренировки, заметки, оборудование) в едином JSON-файле между сеансами работы.

Дополнительно требовалось обеспечить:

- интуитивно понятный интерфейс;
- модульную структуру проекта;
- обработку ошибок ввода и конфликтных ситуаций (занятость тренера, некорректные данные);
- визуализацию данных в виде таблиц и графиков (популярность занятий).

3. Анализ предметной области

Управление фитнес-клубом в ручном режиме (записи в журналах, таблицы Excel) приводит к следующим проблемам:

- трудоёмкость поиска информации о клиенте;
- риск двойной записи на одно время к тренеру
- сложность оперативного подсчёта выручки и популярности услуг;
- отсутствие единого хранилища данных;
- низкая скорость обслуживания клиентов.

Разработанное приложение автоматизирует основные операции:

- регистрация и идентификация по ID (для контроля доступа);
- учёт занятости тренеров с точностью до часа;
- фиксация посещений и автоматическое списание стоимости разовых занятий;
- аналитика выручки и наиболее востребованных занятий;
- управление оборудованием (зал, статус).

4. Выбор средств разработки

В качестве основного языка программирования выбран C++ (стандарт C++11/17), обеспечивающий производительность и богатую экосистему библиотек.

Для создания графического интерфейса использован фреймворк Qt версии 5.12.12. Выбор Qt обусловлен:

- готовыми элементами управления (QWidget, QWidget, QComboBox, QWidget);
- системой сигналов/слотов для обработки событий;
- поддержкой работы с JSON (QJsonDocument, QJsonObject) для хранения данных;
- кроссплатформенностью и бесплатностью (LGPL).

Данные хранятся в JSON-файле, расположенном рядом с исполняемым файлом. Для сериализации используются встроенные средства Qt (QJsonArray, QJsonObject). Дополнительные библиотеки не требуются.

Сборка осуществляется в Qt Creator с использованием компилятора MinGW 64-bit.

5. Проектирование программного обеспечения

При проектировании системы выбрана модульная архитектура, основанная на разделении данных, бизнес-логики и представления.

Класс User – структура для хранения информации о пользователе: логин, пароль, роль, ID (карта), дата окончания абонемента, остатки групповых и персональных занятий, общая сумма потраченных средств. Предоставляет методы toJson() и fromJson() для сериализации.

Класс Equipment – структура для хранения оборудования: название, зал (основной или малый), статус (доступно/в ремонте). Аналогично имеет методы для JSON.

Класс DataStore – центральное хранилище приложения. Содержит векторы пользователей, посещений, покупок, тренировок, заметок тренеров и оборудования. Реализует загрузку/сохранение в JSON (методы load(), save()), CRUD-операции для всех сущностей, а также специализированные методы:

- addVisit(..., double price) – фиксация посещения, автоматическое создание разовой покупки и увеличение выручки;
- isTrainerBusy(), isTrainerBusyExcept() – проверка занятости тренера на интервале 1 час;
- getNextAvailableSlot() – поиск ближайшего свободного времени для тренера.

Класс MainWindow – главное окно приложения. Содержит экземпляр DataStore и информацию о текущем пользователе (currentUser). В зависимости от роли строит соответствующие вкладки с помощью методов buildAdminTabs(), buildTrainerTabs(), buildClientTabs(). Обрабатывает все действия пользователя (добавление/удаление записей, перенос тренировок, покупку абонементов и т.д.).

Вспомогательный класс ComboBoxDelegate – наследует QStyledItemDelegate и используется для отображения выпадающих списков (выбор зала, статуса оборудования) в таблице редактирования оборудования.

6. Описание UML-диаграммы

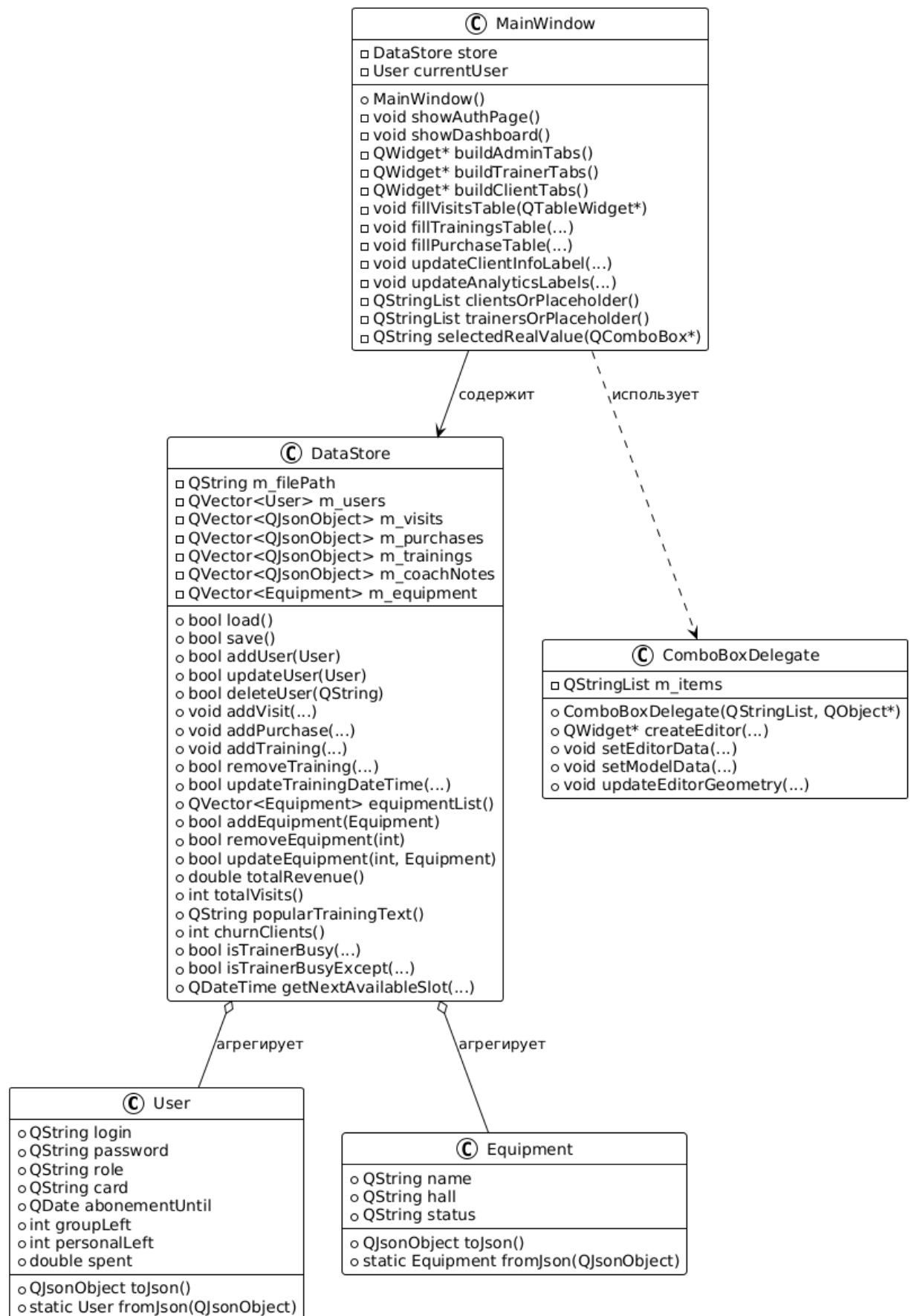


Рисунок 1 — Диаграмма классов. Общая архитектура

Диаграмма демонстрирует структуру программного продукта. Центральный класс MainWindow содержит объект DataStore и текущего пользователя currentUser. Класс DataStore агрегирует контейнеры с объектами User и Equipment, а также хранит временные записи в виде QJsonObject (для посещений, покупок, тренировок, заметок). Класс ComboBoxDelegate используется только в интерфейсе тренера для редактирования таблицы оборудования.

Все методы DataStore, связанные с расчётами (проверка занятости, аналитика), показаны на диаграмме в виде отдельных блоков. Наследование от Qt-классов (QMainWindow, QStyledItemDelegate) отмечено стрелками.

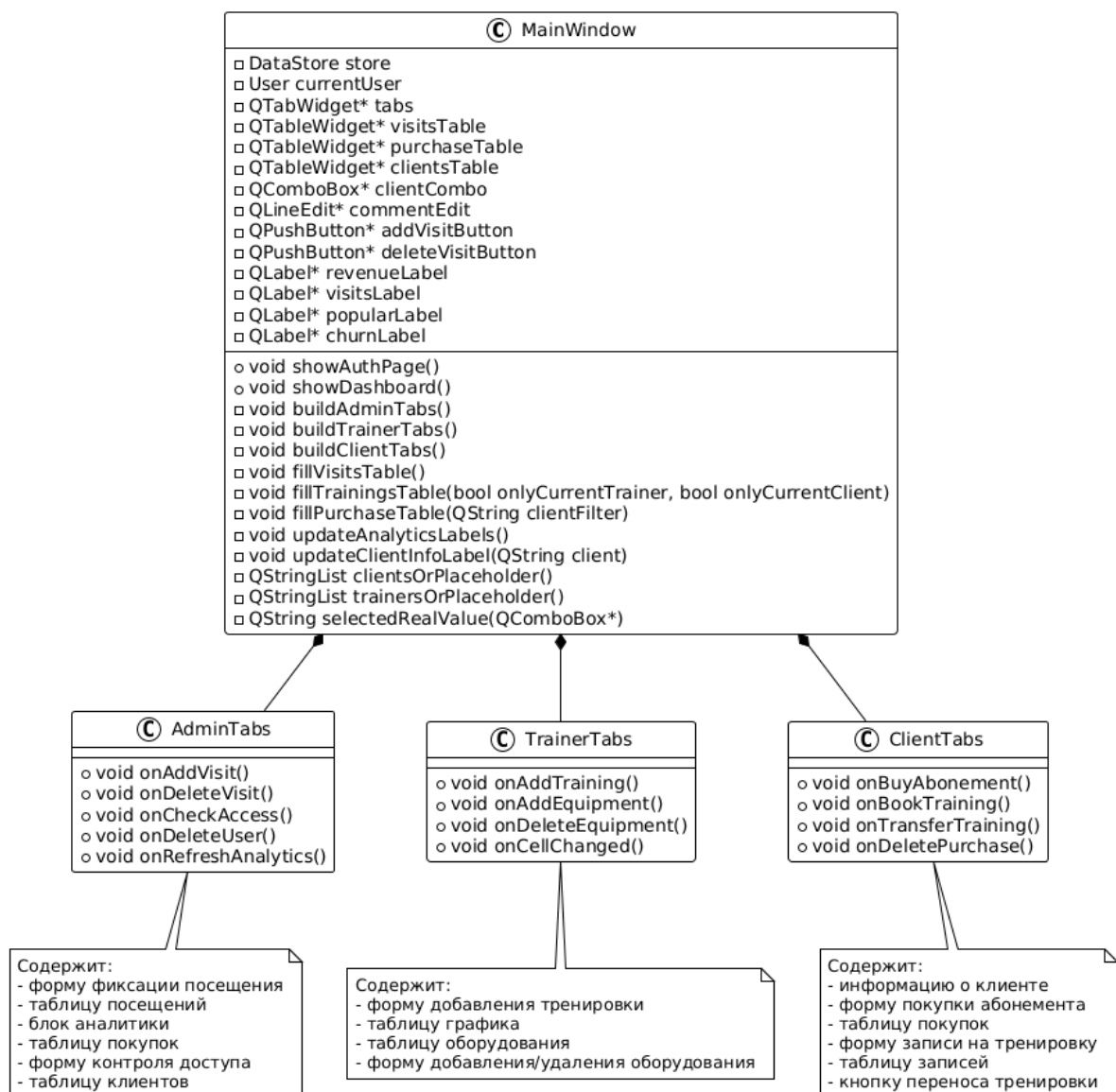


Рисунок 2 — Диаграмма классов. Детальная структура MainWindow и вкладок

Вторая диаграмма раскрывает внутреннее устройство главного окна и его подсистем. Класс MainWindow содержит следующие атрибуты:

- управляющие элементы: store, currentUser;
- виджеты вкладок: tabs (QTabWidget), таблицы visitsTable, purchaseTable, clientsTable;
- элементы формы фиксации посещения: clientCombo, commentEdit, addVisitButton, deleteVisitButton;
- метки аналитики: revenueLabel, visitsLabel, popularLabel, churnLabel.

Основные методы MainWindow разделены на группы:

- авторизация и смена роли: showAuthPage(), showDashboard();
- построение интерфейса: buildAdminTabs(), buildTrainerTabs(), buildClientTabs();
- заполнение таблиц: fillVisitsTable(), fillTrainingsTable(bool onlyCurrentTrainer, bool onlyCurrentClient), fillPurchaseTable(QString clientFilter);
- обновление информации: updateAnalyticsLabels(), updateClientInfoLabel(QString client);
- вспомогательные: clientsOrPlaceholder(), trainersOrPlaceholder(), selectedRealValue().

На диаграмме также выделены три логические подсистемы – AdminTabs, TrainerTabs, ClientTabs – которые соответствуют содержимому вкладок. Для каждой подсистемы перечислены ключевые обработчики (слоты):

- **AdminTabs:** onAddVisit() – фиксация посещения, onDeleteVisit() – удаление выбранного посещения, onCheckAccess() – проверка доступа по ID, onDeleteUser() – удаление клиента, onRefreshAnalytics() – обновление отчёта.
- **TrainerTabs:** onAddTraining() – добавление тренировки в график, onAddEquipment() – добавление нового оборудования, onDeleteEquipment() – удаление выбранного оборудования, onCellChanged() – обработка изменения ячейки таблицы (зал/статус).
- **ClientTabs:** onBuyAbonement() – покупка/продление абонемент, onBookTraining() – запись на тренировку, onTransferTraining() – перенос выбранной тренировки, onDeletePurchase() – удаление собственной покупки.

Все три подсистемы взаимодействуют с MainWindow через его методы и общие данные store, но каждая отвечает за свою область интерфейса. Это обеспечивает модульность и упрощает поддержку кода.

7. Реализация программного средства

Программное средство реализовано в виде однопоточного Qt-приложения. Точка входа – main.cpp, где создаётся объект QApplication, инициализируется главное окно и вызывается show().

Класс DataStore выполняет всю работу с данными. Метод load() читает JSON-файл и заполняет векторы. Если файла нет – создаёт демонстрационный список оборудования. Метод save() сериализует все данные обратно в JSON. Все операции модификации (добавление, удаление, обновление) сопровождаются вызовом save().

Фиксация посещения в методе addVisit() реализована следующим образом:

- если тип тренировки – групповая или персональная, то создаётся разовая покупка (стоимость берётся из таблицы цен), сумма добавляется в spent пользователя и в общую выручку. В поле writeOff таблицы посещений выводится сумма списания.
- для входа без тренировки – списания нет.

Класс MainWindow реализует интерфейс. В showAuthPage() создаются формы входа и регистрации. При регистрации клиента обязательно указывается уникальный ID, иначе регистрация блокируется. После успешной авторизации вызывается showDashboard(), которая строит вкладки в зависимости от роли.

Вкладка администратора содержит:

- форму фиксации посещения (выбор клиента, выбор запланированного занятия из будущих тренировок, комментариев);
- таблицу посещений и кнопку удаления (с проверкой выделения строки);
- блок аналитики (выручка, общее число посещений, популярность занятий, отток клиентов);
- таблицу покупок и кнопку удаления;
- форму контроля доступа (проверка ID или логина, отображение результата);
- таблицу клиентов с ID и сроком абонемента.

Вкладка тренера содержит:

- форму добавления тренировки в график (выбор клиента, типа, занятия, времени) с проверкой занятости тренера и подсказкой свободного слота;
- таблицу графика;
- таблицу оборудования с возможностью редактирования зала и статуса через выпадающие списки (делегат), добавления и удаления записей.

Вкладка клиента содержит:

- форму покупки абонемента, отображение остатков занятий, таблицу своих покупок;
- форму записи на тренировку (тип, занятие, тренер, время) – оплата не происходит, только регистрация в расписании;
- кнопку переноса выбранной тренировки (диалог с выбором нового времени, проверка занятости тренера с исключением текущей записи).

Все таблицы используют кастомный делегат `ComboBoxDelegate` для выпадающих списков (в оборудовании). Для остальных таблиц редактирование запрещено.

8. Основные задачи разработки и принятые решения

В процессе разработки возникли следующие ключевые задачи и были приняты соответствующие решения.

8.1 Организация локального хранения данных

Вместо полноценной базы данных выбран формат JSON. Решение обусловлено простотой реализации, отсутствием внешних зависимостей и возможностью прямого просмотра/редактирования файла. Все операции сериализации выполнены с помощью встроенных средств Qt.

8.2 Проверка занятости тренера

Для предотвращения двойной записи реализована функция `isTrainerBusy()`, которая проверяет пересечение временных интервалов. Дополнительная функция `isTrainerBusyExcept()` используется при переносе, чтобы исключить саму переносимую тренировку из проверки.

8.3 Отображение цен в выпадающих списках

Для удобства клиента при выборе занятия в форме записи строки комбобокса формируются с добавлением цены в скобках, например «Йога (500 Р)». При

смене типа (групповое/персональное) список динамически перестраивается – это реализовано через лямбда-функцию `fillClassCombo`.

8.4 Контроль доступа по ID

Для идентификации клиентов при входе в клуб используется уникальный ID (задаётся при регистрации). Администратор вводит ID или логин, система проверяет активность абонемента или наличие будущих тренировок – если хотя бы одно условие выполнено, доступ разрешается.

8.5 Автоматическое обновление выручки

Деньги за разовые занятия списываются в момент фиксации посещения. При этом создаётся запись в истории покупок и обновляется поле `spent` пользователя. Это позволяет точно отслеживать доходы клуба без ручного ввода.

8.6 Модульность и расширяемость

Выделение `DataStore` как отдельного класса позволило изолировать логику работы с данными. Все операции над пользователями, тренировками и т.д. проходят через этот класс, что упрощает тестирование и добавление новых функций.

9. Тестирование программного средства

Тестирование проводилось путём проверки основных пользовательских сценариев:

- регистрация нового клиента с указанием ID (проверка уникальности);
- регистрация с пустым ID – ошибка;
- вход зарегистрированного пользователя;
- создание тренера, вход под тренером, добавление тренировки в график;
- попытка добавить тренировку на занятое время – вывод предупреждения и подсказка ближайшего свободного слота;
- запись клиента на тренировку, проверка отображения в расписании;
- перенос тренировки клиентом на новое время (при занятости – предложение ближайшего часа);
- фиксация посещения администратором (разовое занятие) – проверка появления записи в истории покупок и увеличения выручки;
- фиксация посещения при наличии абонемента – списания нет, запись о покупке не создаётся;
- удаление посещения, покупки, клиента (только при выделении строки);

- контроль доступа: ввод ID клиента с активным абонементом – доступ разрешён; ввод ID без абонемента и без будущих тренировок – доступ запрещён;
- добавление одинакового оборудования (отсутствие блокировки);
- сохранение и загрузка данных после перезапуска приложения.

Все тесты пройдены успешно. Приложение работает стабильно, ошибки ввода обрабатываются информативными сообщениями.

10. Заключение

В ходе выполнения работы было разработано программное средство «GymControl» – оффлайн система для управления фитнес-клубом.

Разработанная система обеспечивает автоматизацию основных операций:

- регистрация и идентификация клиентов;
- учёт абонементов и разовых занятий;
- построение расписания тренировок с контролем занятости тренеров;
- ведение истории посещений и покупок;
- аналитика выручки и популярности услуг.

В процессе разработки решены задачи:

- проектирования модульной архитектуры с разделением данных и интерфейса;
- реализации локального хранения в формате JSON;
- создания удобного интерфейса с вкладками и таблицами;
- обеспечения устойчивости к ошибкам ввода и конфликтным ситуациям

Использование C++ и Qt позволило создать производительное, кроссплатформенное приложение, которое может быть развёрнуто в любом фитнес-клубе без затрат на интернет и подписки.

Дальнейшее развитие проекта может включать:

- добавление базы данных SQLite для более эффективной работы с большим объёмом данных;
- построение детальных отчётов в формате PDF;
- интеграцию с системой бронирования через веб-интерфейс.

11. Список использованных источников

1. Страуструп Б. Язык программирования C++. — Москва: Бином, 2020.
2. Бланшет Ж., Саммерфилд М. C++ GUI Programming with Qt 4. — Prentice Hall, 2008.
3. Qt Documentation [Электронный ресурс]. — Режим доступа: <https://doc.qt.io/> (дата обращения: 11.05.2026).
4. SQLite Documentation [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения: 11.05.2026).
5. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.
6. Ларман К. Применение UML и шаблонов проектирования. — Москва: Вильямс, 2019.
7. Беликова Е.В., Перфильева И.В. CRM-Системы для фитнес-индустрии: анализ функциональности, удобства использования и проблем внедрения // <https://1economic.ru/> - 2025.
8. 1С:Фитнес клуб. Зачем фитнес-клубу CRM: как выбрать, внедрить и адаптировать систему автоматизации // <https://www.fitness1c.ru/> — 2025.